

Step 5 — An Approval Agent That Only Asks a Human When It Really Needs To — Hints

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper LangGraph `interrupt()`), **Advanced** (what makes the auto-approve path trustworthy instead of a rubber stamp). Read Basic first even if you already know `interrupt()` — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

The Approval agent is the last node, right after the Reasoner. It runs a handful of automatic checks on the draft answer — does it actually reflect the found chunks, does it avoid claiming something risky with no support — and only calls `interrupt()`, LangGraph's built-in pause-for-a-human primitive, when those checks aren't confident. If the checks pass cleanly, it approves on its own and the graph just... continues, no pause at all.

`interrupt()` isn't a crash and it isn't a dead end — calling it inside a node pauses the whole graph right there, using the checkpointer from Step 2 to save exactly where it stopped, and hands control back to whoever's running the graph. A separate, later call resumes it from that exact point once a real human has actually answered.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

The function to build:

```
def approval_node(state: GraphState) -> dict:
```

Run 2-3 concrete checks against the draft (the last message's content) and

`state["found_chunks"]` : does the draft's content plausibly trace back to the chunks (a rough keyword-overlap check, or a small model call asking "is this claim supported by this context, yes/no"), and does the draft avoid language implying an action was actually taken (like "I've sent the refund" vs. "here's what I found about the refund policy" — this project's draft is informational, but a real Approval agent guards this same line before a truly risky action).

If both checks pass: `return {"approved": True}` and let the graph reach `END` normally. If either is unclear: call `interrupt({"draft": draft_content, "found_chunks": [...]})` — this pauses and returns whatever a human later provides as the resume value. Wire `checkpoint=MemorySaver()` (already connected since Step 2) and give every `.invoke()` a `thread_id`, since `interrupt()` /resume depends on it exactly the way Step 2's persistence test did.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

The README's own checklist calls out the real risk directly: "not every run should need a human." A gate whose auto-approval logic is one vague model call ("does this look okay? yes/no") tends to drift toward one of 2 bad extremes over time — either it approves almost everything (the safety feature quietly stops doing anything), or it escalates almost everything (a human becomes a bottleneck on every single run, and the feature gets routed around or ignored). The fix is the same discipline Project 2's Verifier needed: write the auto-approval checks as a literal checklist your code evaluates item by item, not one holistic judgment call — so when something does get wrongly approved or wrongly escalated, you can point at exactly which named check produced that outcome, and fix that one check specifically.

The second real gap: an `interrupt()` that pauses is only half the feature — resuming has to actually continue the graph with the human's real answer, not just re-run from the start. That means your resume script needs to pass the human's decision back in as the *resume value* for the specific `interrupt()` call, using the same `thread_id`, and the node that called `interrupt()` needs to do something different depending on what comes back (approved vs. rejected vs. edited) — not just always proceed once anything at all is returned.

The extra pieces: - `APPROVAL_CHECKS`, a list of named, independently-testable checks (e.g. `check_draft_cites_context`, `check_no_unsupported_action_language`), each returning a pass/fail plus a reason — auto-approve only if every check passes. - A resume script (mirroring Step 2's `resume.py`) that calls `graph.invoke(Command(resume={"decision": "approve"}), config=config)` (or your LangGraph version's equivalent resume mechanism) and confirms the graph actually finishes with that decision reflected in the final state. - At least one test proving the "auto-approves on its own" path (a clean, well-supported draft, zero human involvement) and one proving the "pauses for a human" path (a draft that fails a named check) — same "prove both branches" discipline as every routing decision in this project so far.

Sketch your named checklist (at least 2 checks) before checking Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate get a working `interrupt()` gate that pauses on unclear cases. Advanced is about whether "unclear" is defined precisely enough to stay useful over time — a named checklist that fails loudly and specifically, instead of one vague judgment call that quietly drifts toward always-approve or always-escalate.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
approval_node(state):
    draft = last message's content
    supported = does draft's content roughly match found_chunks?
    risky_language = does draft claim an action was actually taken?

    if supported and not risky_language:
        return {"approved": True}
    else:
        decision = interrupt({"draft": draft, "reason": "needs human review"})
        return {"approved": decision["approved"]}

wire: reason -> approval_node -> end
```

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
agents/approval_agent.py:
    from langgraph.types import interrupt

    def check_draft_cites_context(draft, chunks):
        combined_context = " ".join(c.page_content for c in chunks).lower()
        overlap = any(word in combined_context for word in significant_words(draft))
        return overlap

    def check_no_unsupported_action_language(draft):
        risky_phrases = ["i've sent", "i have issued", "i've refunded", "i have d
        return not any(p in draft.lower() for p in risky_phrases)

    def approval_node(state):
        draft = state["messages"][-1].content
        cites_context = check_draft_cites_context(draft, state["found_chunks"])
        no_risky_language = check_no_unsupported_action_language(draft)

        if cites_context and no_risky_language:
            return {"approved": True}

        decision = interrupt({
            "draft": draft,
            "found_chunks": [c.page_content for c in state["found_chunks"]],
            "failed_checks": [name for name, ok in
                            [ ("cites_context", cites_context), ("no_risky_lang
                                if not ok],
        })
        return {"approved": decision.get("approved", False)}

graph.py:
    builder.add_node("approval", approval_node)
    builder.add_edge("reason", "approval")
    builder.add_edge("approval", END)
```

Write the full typed version yourself, then write `resume.py` (mirroring Step 2's) that resumes a paused run with a human's real decision — before moving to the Advanced version.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

```
APPROVAL_CHECKS = [
    ("cites_context", check_draft_cites_context),
    ("no_risky_language", check_no_unsupported_action_language),
]

approval_node(state):
    draft = ...
    results = {name: check(draft, ...) for name, check in APPROVAL_CHECKS}
    if all(results.values()):
        return {"approved": True}
    failed = [name for name, ok in results.items() if not ok]
    decision = interrupt({"draft": draft, "failed_checks": failed})
    return {"approved": decision.get("approved", False)}
```

Here's almost the whole thing — fill in the missing piece yourself:

```
APPROVAL_CHECKS = [
    ("cites_context", check_draft_cites_context),
    ("no_risky_language", check_no_unsupported_action_language),
]

def approval_node(state):
    draft = state["messages"][-1].content
    results = {}
    for name, check in APPROVAL_CHECKS:
        # your turn: each check function takes different arguments (draft alone,
        # or draft + found_chunks) – call each correctly and store its bool result
        ...

    if all(results.values()):
        return {"approved": True}

    failed = [name for name, ok in results.items() if not ok]
    decision = interrupt({"draft": draft, "failed_checks": failed})
    return {"approved": decision.get("approved", False)}
```

```
# your turn: write test_auto_approves_clean_draft() and  
# test_pauses_for_unsupported_draft(), proving both branches
```

Fill in the marked piece and both tests, then compare against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate get a real `interrupt()` gate wired in with 2 concrete checks. Advanced makes those checks a named, independently callable list instead of 2 variables inline in the function body — the difference between "something failed" and " `no_risky_language` specifically failed, here's why," which is what a real deployment needs when deciding whether to trust the gate or tighten it.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)